

ECE391
Computer System Engineering
Lecture 8

University of Illinois at Urbana-Champaign

Spring 2024

Lecture Topics

- Synch issues
- Conservative synchronization design
- Semaphores
- Reader/writer synchronization

Spin Locks Recap

- Protect critical sections from other CPUs
- Just a variable with an atomic test-and-set operation

```
int trylock(int * ptr);  
void lock(int * ptr) {  
    while (trylock(ptr))  
        continue;  
}  
void unlock(int * ptr) {  
    *ptr = 0;  
}
```

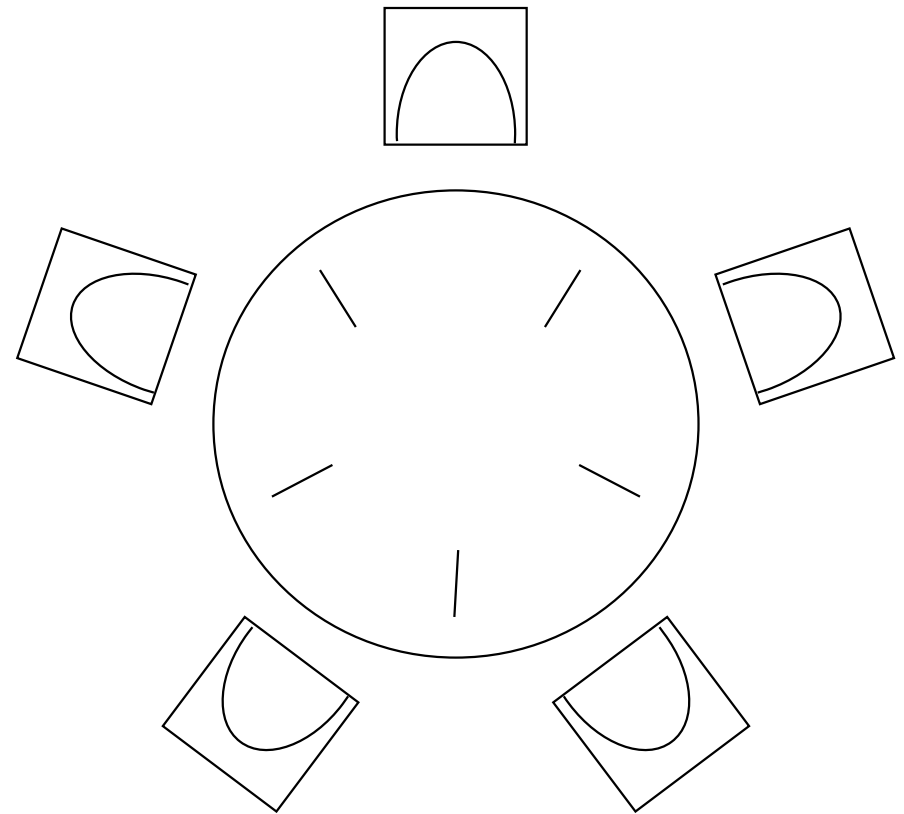
```
_trylock:  
    pushl    %ebp  
    movl     %esp, %ebp  
    pushl    %ebx  
    movl     8(%ebp), %ebx  
    movl     $1, %eax  
    xchgl    %eax, (%ebx)  
    popl     %ebx  
    movl     %ebp, %esp  
    popl     %ebp  
    ret
```

ARM Atomic Operations

- **LDREX** Rt, [Rn, imm8]
Load Rt from memory address Rn+imm8 and reserves exclusive access to the location
- **STREX** Rd, Rt, [Rn, imm8]
Attempt to store Rt to memory address Rn+imm8, set Rd=0 if successful, Rd=1 if not successful
 - Store succeeds if no other access to address
- Test-and-set:
 1. Load from lock address using LDREX
 2. If lock value is 0, attempt to store 1 using STREX
 3. If store fails (Rd=1), repeat from step 1

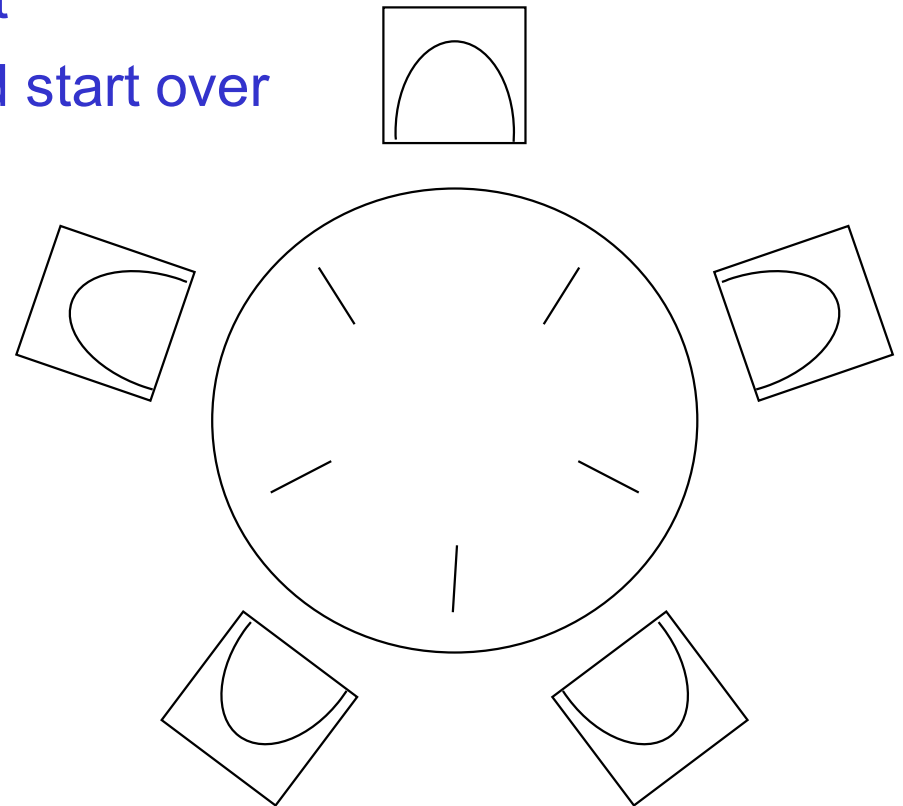
The Dining Philosophers

- Synchronization issues
 - five hungry philosophers eating noodles
 - five chopsticks
- Protocol
 - take left chopstick (or wait)
 - take right chopstick (or wait)
 - eat
 - release right chopstick
 - release left chopstick
 - digest
 - repeat



Another Philosophy Lesson (cont.)

- How about the following protocol?
 - take left chopstick (or wait)
 - if right chopstick is free, take it
 - else release left chopstick and start over
 - eat
 - release right
 - release left
 - digest
 - repeat
- Does this work?



Another Philosophy Lesson (cont.)

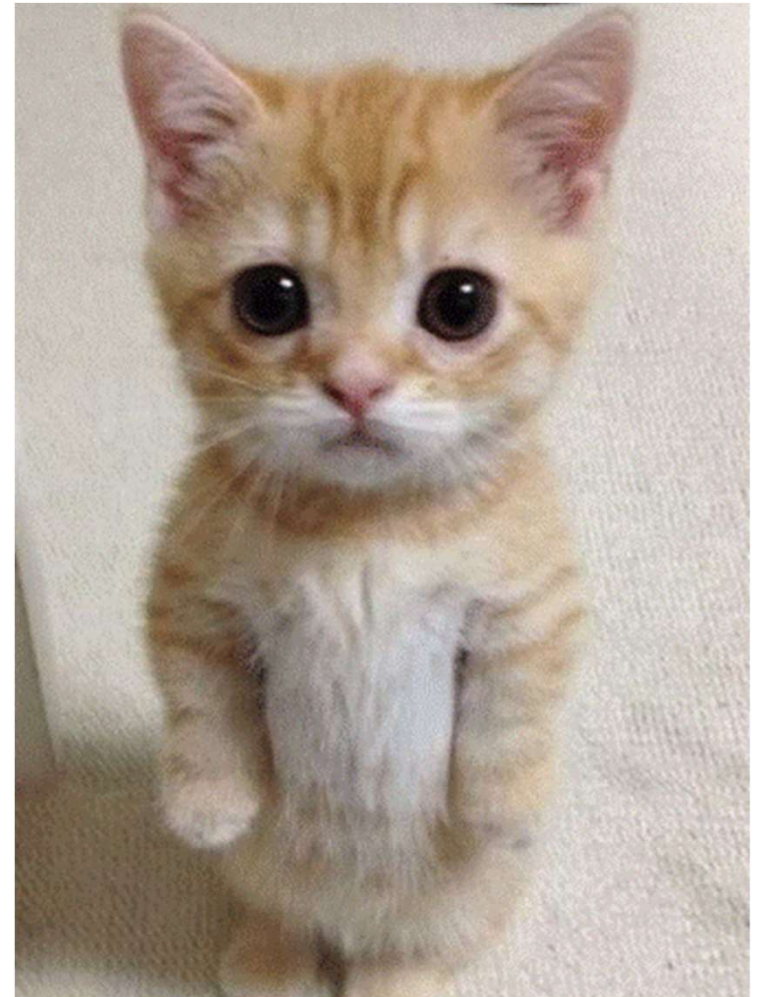
- What if all philosophers act in lock-step (same speed)?

left	left	left	left	left
release	release	release	release	release
left	left	left	left	left
release	release	release	release	release
left	left	left	left	left

(ad infinitum)

Deadlock is Deadly

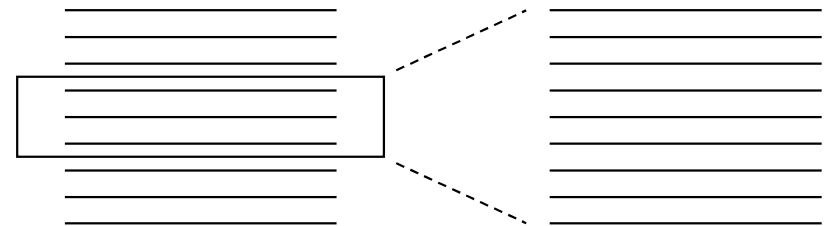
- *Deadlock makes kittens sad*
- Much research on avoiding deadlocks
- Some systems can detect deadlock, but this is not a fix; only useful to debug
- Simple strategy:



Another Philosophy Lesson (cont.)

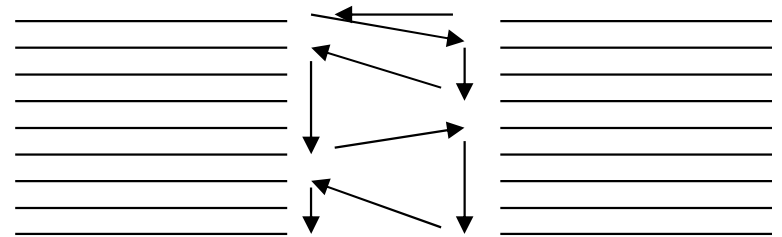
Conservative Synchronization Design

- Getting synchronization correct can be hard
 - it's the focus of several research communities
- On uniprocessor
 - mentally insert handler code between every pair of adjacent instructions
 - ask whether anything bad can happen
 - if so, prevent with CLI/STI



Conservative Synchronization Design (cont)

- On a multiprocessor
 - consider all possible interleavings of instructions
 - amongst all pieces of (possibly concurrent) code
 - ask whether anything bad can happen
 - if so, use a lock to force serialization
 - good luck!



Conservative Synchronization Design (cont)

- What does “bad” mean, anyway?
- A conservative but systematic definition
 - if any data written by one piece of code
 - are also read or written by another piece of code
 - these two pieces must be atomic with respect to each other

Conservative Synchronization Design (cont)

not shared

- What variables are shared?
- *step 0*: ignore the parts that don't touch shared data
- *step 1*: calculate read & write sets
- *step 2*: check for R/W, W/W relationships
 - **must be atomic!**
- *step 3*: add lock(s) to guarantee atomic execution (pick order if > 1 locks)
- *step 4*: optimize if desired

```

typedef struct {
    double mass;
    double x, y, z;    /* position */
    double vx, vy, vz; /* velocity */
} thing_t;

void move(thing_t* t)
{
    t->x += t->vx;
    t->y += t->vy;
    t->z += t->vz;
}

double dist(thing_t* t)
{
    return sqrt(t->x * t->x +
               t->y * t->y +
               t->z * t->z);
}

double speed(thing_t* t)
{
    return sqrt(t->vx * t->vx +
               t->vy * t->vy +
               t->vz * t->vz);
}

double momentum(thing_t* t)
{
    double tmp = t->mass;
    return tmp * speed(t);
}

void stop(thing_t* t)
{
    t->vx = t->vy = t->vz = 0;
}

void change_mass(thing_t* t, double new_mass)
{
    t->mass = new_mass;
}

```

```

#include<stdio.h>

typedef struct person_t person_t;
struct person_t {
    char*    name;
    int      age;
    person_t* next;
};

static person_t* group;

void birthday(person_t* p)
{
    p->age++;
}

void list_people(void)
{
    person_t* p;
    int num;

    for (p=group, num=0; NULL != p; p=p->next, num++)
    {
        if (10 > num)
            printf("%s %d\n", p->name, p->age);
        else
            printf("%s\n", p->name);
    }
}

```

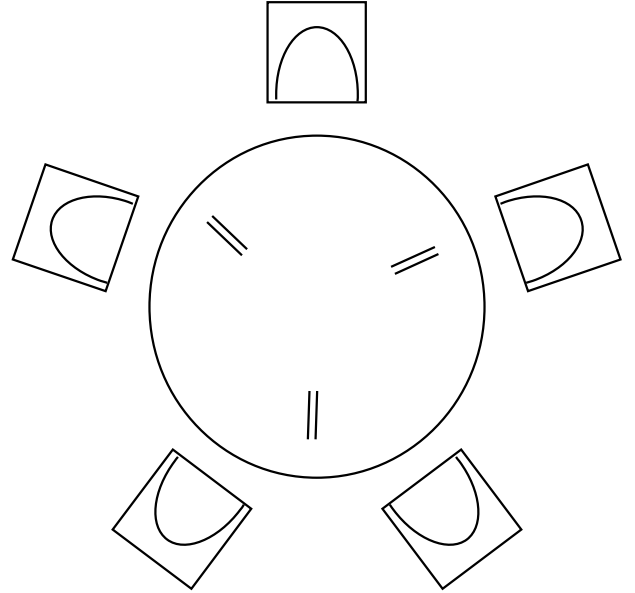
Conservative Synchronization Design – Example Code Analysis

- Read/write sets

Conservative Synchronization Design – Example Code Analysis (cont)

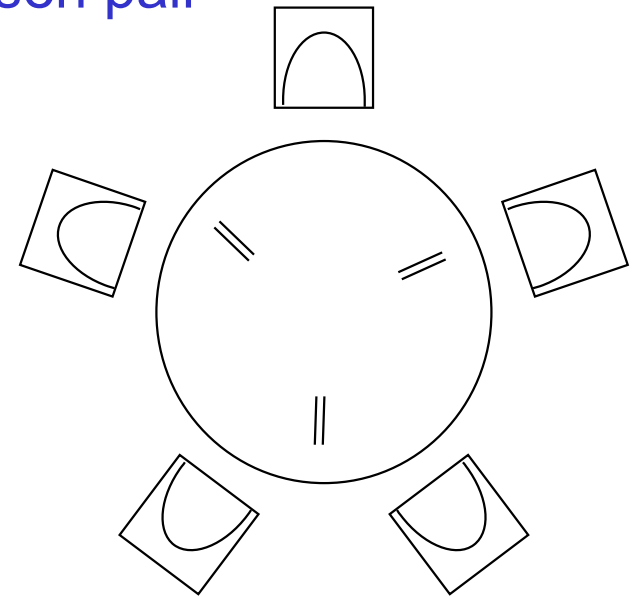
- Each lock = circle in graph
- All edges must be contained in some circle
- One lock suffices, but prevents parallelism (performance)
- Could use three (as shown above);
then MUST pick a lock order!

Role of Semaphores

- Recall our philosophical friends
 - Five philosophers
 - Three pairs of chopsticks (one “lock” per pair)
 - Problem: how do you get a pair?
- 
- The diagram shows a circular table with five square plates, each containing a semi-circle representing a philosopher, arranged around it. Three pairs of chopsticks are placed on the table, each pair consisting of two parallel lines. The chopsticks are positioned between the plates: one pair is at the top, one at the bottom, and one on the right side. The left side of the table is empty.
- Option 1: walk around the table until you find a pair free
 - lots of walking
 - other people may cut in front of you

Role of Semaphores

- Option 2: pick a target pair and wait for it to be free
 - other pairs may be on the table
 - but you're still waiting hungrily for your chosen pair
- Instead, use a **semaphore**!
 - an atomic counter to track number of free chopstick pairs
 - To pick up a pair, philosopher tests if counter is non-zero and (if non-zero) decrements it
 - Operations must be atomic



Semaphore Operations

- **Semaphore** is a counter with special operations
 - Usually used to count number of available resources
- **DOWN** also called *test/decrement* or *proberen*
- **UP** also called *increment* or *verhogen*

Semaphores

- When are semaphores useful?

Linux Semaphore API



```
void sema_init (struct semaphore* sem, int val);
```

Initialize dynamically to a specified value

```
void init_MUTEX (struct semaphore* sem);
```

Initialize dynamically to value one (mutual exclusion)

```
void down (struct semaphore* sem);
```

Attempt to decrement, wait if zero (P operation)

```
void up (struct semaphore* sem);
```

Increment the semaphore (V operation)

Mutexes



- Locks are also called mutexes (mutual exclusion)
- Semaphore with initial value 1 can be used as mutex
- Some systems may have one or the other
 - pthreads (POSIX threads) has mutexes, not semaphores
 - System V IPC has semaphores, not mutexes
- Linux also has straight mutexes (not semaphores)
 - `mutex_lock(struct mutex * lock)`
 - `mutex_unlock(struct mutex * lock)`

Mutexes vs Semaphores

- Two very similar synchronization primitives
- Mutexes and locks have a concept of owner
 - We say mutex/lock “held” by someone
 - Can be easier to reason about
- Semaphores are an atomic counter
 - No inherent ownership
 - In principle can call up() without down()

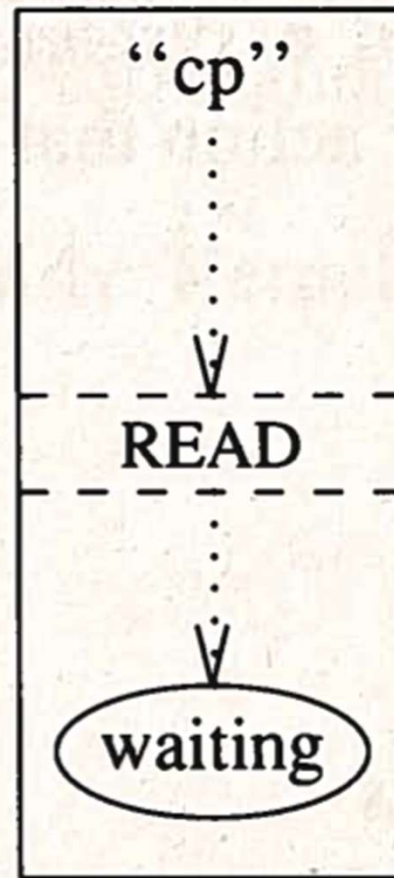
```
struct mutex {  
    atomic_t count;  
    spinlock_t wait_lock;  
    struct list_head wait_list;  
}
```

```
struct semaphore {  
    spinlock_t lock;  
    unsigned int count;  
    struct list_head wait_list;  
};
```

Linux Spinlocks vs Mutexes

- Mutexes and semaphores put waiting process to sleep
 - Can only be used in process context (handling system call)
 - Must **not** be used in interrupt context (interrupt handler)
 - What does “put process to sleep” mean? *Context switch*
 - We will talk about this a lot more later, I promise!
- Spinlocks “spin” while waiting
 - May be used in interrupt context
- Use mutexes for long critical sections
 - Or semaphores in role of mutex

User Process

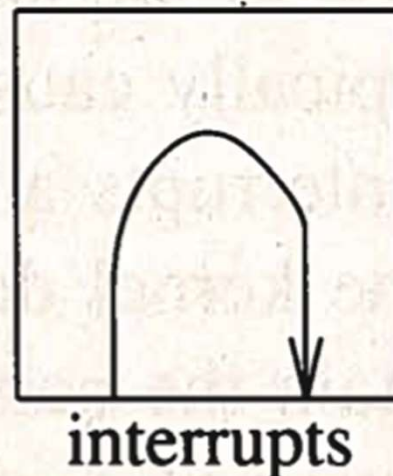


Preemptive scheduling;
cannot block;
runs on user stack in
user address space

Top Half
of Kernel

Runs until blocked or done;
can block to wait for resource;
runs on kernel stack in
user address space

Bottom Half
of Kernel



Never scheduled;
cannot block;
runs on kernel stack in
kernel address space

Reader/Writer Problem: The Philosophers and Their Newspaper

- Philosophers like to read newspaper
 - each philosopher reads frequently, taking short breaks between
 - multiple philosophers may read at same time
 - different sections or or just over another's shoulder
- Paper carrier delivers new paper
 - once per day (infrequently)
 - must change all sections at once
- Reader/writer synchronization supports this style
 - allows many (in theory infinite) readers simultaneously
 - at most one writer (and never at same time as readers)
- What if newspaper is always being read? starvation!

Reader/Writer Locks in Linux

- Linux provides two types of reader/writer synchronization
 - reader/writer spin locks (`rwlock_t`)
 - extension of spin locks
 - use for short critical sections
 - ok to use in interrupt handlers
 - admit writer starvation (you must ensure that this not happen)
 - reader/writer semaphores (`struct rw_semaphore`)
 - Mutex-like (sleep waiting) analog or `rwlock`
 - Use only in process context (not interrupt handlers)
 - No writer starvation (new readers wait if writer is waiting)